Trusted Code Generation: A Formal Trust-Boundary Framework for Specification-Conditioned Smart-Contract Synthesis and Verification

Version 3: Proof-Carrying Bundles for Critical Infrastructure

Vladimir Stoliarov

October 29, 2026

Abstract

Large language models (LLMs) can generate smart-contract code from informal human intent, but their outputs carry no formal guarantee. *Trusted Code Generation* (TCG) draws an explicit trust boundary between an untrusted generation stage, which may produce arbitrary or adversarial output, and a trusted, formal verification core built on a sound formal semantics of the Ethereum Virtual Machine (EVM). The design claim is that system soundness is invariant to the capabilities of the generative model. Version 3 turns the pipeline into a portable unit of trust: the proof-carrying bundle. Version 2 stopped at returning a pair (b, φ); the certificate that justified acceptance was consumed by the checker and discarded. Version 3 keeps it. A bundle packages the artifact, the specification, the certificate, the instantiated trusted base and the evidence of non-vacuity into one content-addressed object that a third party re-verifies with a small checker, offline, with no access to the session that produced it. The framework becomes parametric in the domain: any domain that supplies a formal semantics, a specification language over its configurations, a sound certificate checker and an identity function for deployed artifacts admits a bundle; the EVM is one instantiation. The bundle separates three layers with different kinds of trust (a hard layer checked by a certificate checker, an evidence layer judged by the human oracle, and a provenance layer that is informative only), and a verdict ladder L0–L4 reports each separately, so that a machine-checked proof is never conflated with a well-documented opinion. The one non-formal gap, the alignment between the formal specification and the human's intent, stays with a human; the bundle gives that human tools and a signature, never a replacement.

Note on this version. Part I (Sections 1–9) reproduces version 2 of this paper (27 June 2026) verbatim; only the heading of Section 9 differs. Known errors in Part I are listed, not corrected, in Section 21. Part II (Sections 10–21) is new and contains the bundle. A companion note, *Specification Games* [9], develops the mutant witnesses of Section 16 into a certified score and studies what a specification-writing generator does to it; it is referenced where relevant and not reproduced.

Part I
The Trust Boundary (version 2)

1 Introduction

The translation of informal human intent into executable code is now routinely performed by large language models. In the smart-contract setting this is attractive and dangerous in equal measure: contracts

are short, their execution environment is deterministic, and they custody real value, so an undetected defect is directly monetizable by an adversary. An LLM that emits Solidity from a natural-language description offers speed, but the artifact it produces is *plausible*, not *proven*. In the vocabulary of epistemology, the model supplies neither justification nor a guarantee of truth; it supplies a high-likelihood sample from a learned distribution over token sequences.

A natural reaction is to make the generator more reliable—better training, more reasoning, self-critique. We argue this is the wrong axis. No amount of generative competence converts a statistical sample into a proof, because the two belong to different categories. Instead, TCG accepts the generator as permanently untrusted and relocates the guarantee to a component that can actually carry it: a sound verifier operating over a formal semantics of the target machine. The generator (informally, "vibe coding") is then free to be wrong as often as it likes; its errors are caught, not trusted.

Scope. TCG applies precisely when the domain admits a formal semantics. Smart contracts on the EVM are a near-ideal instance: the EVM is deterministic once its execution environment is fixed (Section 3.1) and its execution is bounded by gas, and a complete executable formal semantics, KEVM [1], exists and is validated against the official Ethereum test suite. Tasks without a formal notion of success—"design a profitable business"—fall outside the framework entirely, not because verification is expensive there but because no correctness predicate exists to verify against. The boundary of applicability is exactly the boundary of formalizability.

Contributions.

1. A three-layer formalization—operational checker, semantic truth, and a bridging soundness assumption—that makes the soundness theorem a genuine result rather than a consequence of how the verifier is defined (Sections 3.4 and 5).
2. A pipeline model assigning an explicit trust status—untrusted (U), trusted-formal (T), or non-formal trusted oracle (T^*)—to every component, with the human kept outside the formal core (Section 3.3).
3. A specification model over full machine configurations—precondition, trace invariant, postcondition, and exit status—so that trace-sensitive security properties are expressible and covered by the theorem (Section 3.2).
4. A completeness corollary that formalizes the specification blind spot and reframes specification completeness, with an explicitly heuristic coverage proxy, as the central open problem (Section 6).

2 Background and Setting

2.1 The barrier to a verified translator

It is tempting to seek a *provably correct* translator from natural language to formal specification, analogous to a verified compiler such as CompCert [2]. This is impossible in principle, and the impossibility is categorical rather than engineering. A compiler can be proven correct because both endpoints carry a formal semantics: one proves $[\![src]\!] = [\![compile(src)]\!]$. Natural language has no formal semantics. There is nothing to place on the left-hand side of such an equation, so correctness of translation is not a well-defined predicate. The consequence for system design is decisive: the goal cannot be to *trust* the translation; it can only be to make an incorrect translation *detectable*, and to render the residual, undetectable gap explicit. This posture is reinforced by empirical findings in autoformalization, where minor paraphrases of an identical natural-language statement produce materially different formal outputs [4], and where the alignment between an informal statement and its formalization must itself be evaluated rather than assumed [5].

2.2 A semantics-first verification target

We assume a sound, executable formal semantics of the EVM that defines a transition relation over machine configurations, validated for faithfulness against a reference implementation by an extensive conformance suite [1]. Independent formalizations of EVM semantics in other systems—for example in Dafny [3]—make cross-validation of a critical property feasible and provide leverage against the risk that any single semantics mismodels the machine. Verifying at the level of deployed bytecode—rather than source—removes the source compiler from the trusted base: whatever the compiler emits, the verifier reasons about the actual deployed artifact. This choice determines where the trust boundary can be drawn and is assumed throughout.

3 The TCG Pipeline

3.1 Machine configurations and the environment

The EVM is deterministic only once its execution environment is fixed. We therefore take states to be full configurations rather than abstract points.

Definition 1 (Configurations)**.** Let Σ be the set of full EVM/world-state configurations, each including account state, persistent storage, contract code, balances, the transaction parameters (caller, calldata, value), block parameters, remaining gas, the active fork rules and fee schedule, precompile semantics, and the code of any reachable external contract. Let $\mathscr{E}$ denote an *environment class*: a set of configurations agreeing on fork rules, fee schedule, and the deployment assumptions on caller, calldata shape, and external code. The real machine induces a transition relation $\to_{\mathrm{EVM}}^{f,g}$ parameterized by fork f and fee schedule g; the formal semantics induces $\to_{K}^{f,g}$ likewise.

A *trace* is a finite sequence of configurations $\sigma_0 \to \sigma_1 \to \cdots \to \sigma_m$ together with the externally observable events emitted along it (logs, external calls, value transfers) and the terminating *exit status* $e \in \{\mathsf{halt}, \mathsf{revert}, \mathsf{oog}, \mathsf{exc}\}$ (successful halt, revert, out-of-gas, exceptional halt).

3.2 Specifications over configurations

To express trace-sensitive properties—reentrancy, callback discipline, event emission, and the success/revert distinction—a specification cannot be a pair of state predicates alone.

Definition 2 (Specification)**.** A specification is a tuple $\varphi = (\mathrm{Pre}, \mathrm{Trace}, \mathrm{Post}, \mathrm{Exit})$, where Pre and Post are predicates over configurations, Trace is a predicate over traces (a trace invariant, e.g. "no external call re-enters this contract before its first invocation returns"), and $\mathrm{Exit} \subseteq \{\mathsf{halt}, \mathsf{revert}, \mathsf{oog}, \mathsf{exc}\}$ constrains the admissible exit statuses. A trace τ from σ_0 *satisfies* φ, written $\tau \models \varphi$, if $\sigma_0 \models \mathrm{Pre}$ implies that $\mathrm{Trace}(\tau)$ holds, the exit status of τ lies in Exit, and the terminal configuration satisfies Post. The partial-correctness terminal claim is the special case Trace $\equiv$ *true*, Exit $= \{\mathsf{halt}\}$.

3.3 Components and trust status

Let C be the set of proof certificates and $A = A_{\mathrm{chk}} \uplus A_{\mathrm{asm}}$ the set of annotations, partitioned into those the checker fully discharges as obligations (A_{chk}: loop invariants, ranking functions, predicate strengthenings) and those it would otherwise consume as unverified facts (A_{asm}: e.g. storage-layout hints). Components and their trust status:

$F : N \to S \cup \{\bot\}$	$[U]$	formalizer
$G : S \times CE^* \to B$	$[U]$	generator
$I : S \times B \to A \times C$	$[U]$	synthesizer
$\mathrm{Check} : S \times B \times A \times C \to \mathrm{Res}$	$[T]$	checker
$H : N \times S \to \{\mathsf{ok}, \mathsf{rej}\}$	$[T^*]$	human oracle

where the checker's result type is structured,

$$\mathrm{Res} = \mathsf{Accept}(C) \uplus \mathsf{Refute}(CE) \uplus \mathsf{Unknown},$$

distinguishing a discharged proof, a concrete counterexample trace, and a failure to decide (timeout, solver incompleteness). The marker T^* denotes *non-formal trusted*: H is the sole bearer of the semantics of N, belongs to the trusted base but *not* to the formal core, and cannot be replaced by any formal artifact.

Remark 1 (The two trusted parts are distinct). The *formal trusted core* consists of the checker together with the proof checker and the formal EVM semantics:

$$\text{formal core} = \{\text{Check},\ \text{proof checker},\ \text{formal EVM semantics}\}.$$

The *non-formal trusted assumption* is H. The abstract's "trusted formal core" refers only to the former; the human is a trusted *assumption*, not a verification component. Components F, G, I are untrusted and may behave arbitrarily, including adversarially. In particular the certificate and the checked annotations A_{chk} are *inputs* to Check, never axioms; the checker does not believe an invariant, it discharges it. Annotations in A_{asm}, by contrast, are not self-checking and must be either eliminated or promoted to an explicit member of the trusted base (Assumption 5).

3.4 The verification predicate: three layers

We deliberately separate the operational checker from semantic truth, so that the bridge between them is an assumption and not a definition.

Definition 3 (Operational acceptance)**.** $\mathsf{Check}(\varphi, b, a, c) = \mathsf{Accept}(c)$ holds iff the checker mechanically validates the certificate c as discharging every proof obligation that the claim "b satisfies φ over $\to_K$" generates under annotations a. This is a decidable, syntactic procedure on c; it makes no reference to truth.

Definition 4 (Semantic claim)**.** Write $\to_K^{f,g} \models \varphi(b)$ for the semantic statement that, in the formal semantics under fork f and fee schedule g, every terminating $\to_K^{f,g}$-trace of b from a configuration satisfying Pre satisfies φ in the sense of Definition 2 (trace invariant, exit status, and postcondition).

These two are linked only through the soundness of the checker, stated as Assumption 3 below; neither implies the other by definition. The value $\mathsf{Unknown}$ and any $\mathsf{Refute}(\cdot)$ are both treated as non-acceptance: an obligation that is not discharged is not accepted. Only $\mathsf{Refute}(ce)$ carries a counterexample that can be fed back to the generator; $\mathsf{Unknown}$ carries none.

3.5 The process

Definition 5 (TCG)**.** For intent $n \in N$, the process proceeds stepwise (avoiding any ill-typed application of H to $\bot$):

1. If $F(n) = \bot$, return $\bot_{\mathrm{spec}}$.
2. Otherwise let $\varphi = F(n)$. If $H(n, \varphi) = \mathsf{rej}$, return $\bot_{\mathrm{spec}}$.
3. Otherwise iterate for $k = 1, \ldots, K_{\max}$: let $b_k = G(\varphi, ce_{<k})$ and $(a_k, c_k) = I(\varphi, b_k)$.
   If $\mathsf{Check}(\varphi, b_k, a_k, c_k) = \mathsf{Accept}(c_k)$, return (b_k, φ).
   If the result is $\mathsf{Refute}(ce_k)$, append ce_k to the feedback and continue.
   If the result is $\mathsf{Unknown}$, continue with no feedback.
4. If no iteration accepts, return $\bot_{\mathrm{gen}}$.

The process has three outcomes, all safe: abstention at the specification stage ($\bot_{\mathrm{spec}}$), failure to synthesize an accepted artifact within budget ($\bot_{\mathrm{gen}}$), or an accepted pair (b, φ). The system never emits a non-accepted artifact labeled as accepted; this is the soundness of the *construction*, prior to any environmental assumption.

4 Assumptions (Trusted Base)

The guarantee holds relative to an explicit trusted base T.

Assumption 1 (Model adequacy)**.** $\to_K^{f,g}$ agrees with $\to_{\mathrm{EVM}}^{f,g}$ on the target fork f and fee schedule g, over the environment class $\mathscr{E}$. Status: *validated* against the official conformance suite [1], not proved.

Assumption 2 (Environment adequacy)**.** The environment encoded in Σ and $\mathscr{E}$ matches deployment: caller, calldata class, external contract code, precompiles, block parameters, chain id, balances, gas limits, and fork version.

Assumption 3 (Checker soundness)**.** The checker and its proof/SMT backend are sound: if $\mathrm{Check}(\varphi, b, a, c) = \mathsf{Accept}(c)$ then $\to_K^{f,g} \models \varphi(b)$. This is the bridge between Definitions 3 and 4.

Assumption 4 (Artifact identity)**.** The verified bytecode is bit-identical to the deployed runtime bytecode, accounting for constructor versus runtime code and immutables.

Assumption 5 (Annotation discipline)**.** Every annotation used by Check lies in A_{chk} and is discharged as an obligation; any A_{asm} fact (e.g. storage layout) is either eliminated or listed explicitly in T with an independent justification.

Assumption 6 (Specification well-formedness)**.** Pre is satisfiable and describes reachable deployment configurations, so that the guarantee is non-vacuous.

Assumption 7 (Human validation)**.** $H(n, \varphi) = \mathsf{ok}$, and the human accepts the contract that *the artifact will be verified exactly against* φ: the specification is declared the oracle of correctness. This is a trusted *assumption*, not a formal step.

Assumption 7 is the methodological core. It converts the undecidable claim "φ matches intent" into an explicit, shared premise. We do not assert that $\varphi \approx n$ is provable—it is not. We make the dependence on a human judgment a visible part of the theorem rather than a hidden defect.

5 Soundness

Theorem 1 (Soundness of TCG)**.** *Under Assumptions 1–7, if the process returns* (b, φ)*, then for every real EVM trace* τ *of the deployed bytecode* b*, under fork* f *and fee schedule* g*, starting from any reachable configuration satisfying* Pre*, the trace satisfies* φ*: its events obey the trace invariant* Trace*, its exit status lies in* Exit*, and its terminal configuration satisfies* Post*.*

Proof. The process returns (b, φ) only via $\mathrm{Check}(\varphi, b, a, c) = \mathsf{Accept}(c)$ for some checked annotations $a \in A_{\mathrm{chk}}$ and certificate c (Definition 5, Assumption 5). By Assumption 3, operational acceptance entails the semantic claim $\to_K^{f,g} \models \varphi(b)$ (Definition 4)—this step is where soundness is used and is not available by definition. By Assumptions 1 and 2, $\to_K^{f,g}$ coincides with $\to_{\mathrm{EVM}}^{f,g}$ over $\mathscr{E}$, transferring the claim to the real machine. By Assumption 4, b is the deployed artifact, and by Assumption 6 the precondition is satisfiable and reachable, so the quantification is non-empty. Hence every real EVM trace of b from a reachable Pre-configuration satisfies φ in the sense of Definition 2. □

Remark 2 (What the theorem does not claim). The theorem establishes correctness *relative to* φ, not relative to intent: returning (b, φ) does not entail $b \approx n$. The gap between φ and n is closed solely by the human oracle H (Assumption 7), never by the mathematics. This is the single *non-formal* gap; the remaining gaps (model adequacy, checker soundness, artifact identity, environment) are formal and discharged as Assumptions 1–4.

Proposition 1 (Capability-invariance, with non-vacuity caveats)**.** *The conclusion is independent of F, G, and I: replacing any with an arbitrary or adversarial procedure leaves soundness intact; only liveness is affected. Two caveats bound the* usefulness *of the guarantee, not its soundness: an unsatisfiable* Pre *would make the claim vacuous (excluded by Assumption 6), and a too-weak* (Trace, Post, Exit) *would make it true but uninformative. Non-vacuity therefore requires, beyond Assumption 6, external evidence that φ is strong enough—e.g. that known-bad contracts are refuted and known-good traces admitted.*

Proof. None of Assumptions 1–7 references F, G, or I; the hypothesis already encodes that Check accepted, so the producer of b, a, c is immaterial to checker soundness. The caveats follow because Definition 2 is satisfied vacuously when Pre is unsatisfiable and trivially when Trace, Post are weak. □

6 Completeness and the Specification Blind Spot

Soundness says an accepted artifact satisfies φ. It says nothing about properties absent from φ.

Corollary 1 (Blind spot)**.** *Let closure(φ) be the set of properties entailed by φ. For any $\psi \notin$ closure(φ), the process makes no claim about the behavior of b with respect to ψ. In particular, acceptance does not entail the absence of defects outside φ.*

The corollary formalizes a single sentence: *silence in the specification is a blind spot, not a guarantee of safety*. The configuration-level specification of Definition 2 widens what *can* be stated—reentrancy and callbacks via Trace, the success/revert/out-of-gas distinction via Exit, emitted logs as trace events—but each must still be written down to be guaranteed. Cross-contract invariants and storage collisions under delegated calls remain outside any single-contract φ unless modeled explicitly.

This relocates the central research problem. The verifier and the machine semantics are strong; the weak link is the *completeness* of φ. The productive role of generative models inside the framework is therefore not to be trusted with code, but to assist in producing a *more complete* specification—proposing candidate properties and surfacing unspecified paths—while remaining untrusted: every proposed property is validated by H and checked by Check.

Definition 6 (Operational coverage heuristic)**.** Let $\mathscr{P}(b)$ be the set of feasible symbolic execution paths of b under a fixed exploration strategy and let $cov_\varphi(b)$ be the subset on which φ constrains trace, exit status, or terminal configuration. The ratio $|cov_\varphi(b)|/|\mathscr{P}(b)|$ is a *heuristic*, not a completeness metric: $\mathscr{P}(b)$ depends on abstraction and solver strategy, a covered path may be covered by a trivial constraint, and path coverage is not semantic-property coverage. It is reported as one indicator among several (mutation score, obligation coverage, exit-status coverage, arithmetic-boundary coverage, and an adversarial-property checklist), never as a guarantee of completeness.

7 Discussion

7.1 The role of the invariant synthesizer

Closing a reachability claim over code with loops requires loop invariants, whose discovery is undecidable in general. The Hoare-logic treatment of a loop replaces an unbounded execution by three obligations on a candidate invariant—establishment on entry, preservation by the body, and sufficiency on exit [6]. TCG exploits the resulting asymmetry: although *finding* an invariant is hard, *checking* a proposed invariant reduces to these obligations, each dischargeable by the backend. The synthesizer may therefore guess freely. A wrong invariant fails preservation and is refuted; a too-weak invariant fails the exit obligation and is refuted; only a correct invariant closes the claim, and then by construction. This is the same untrusted-proposal/trusted-check pattern that governs G, applied to annotations in A_{chk}. It does *not* extend to A_{asm}, which is why storage-layout facts are handled by Assumption 5 rather than absorbed silently.

7.2 Threats to the trusted base

The assumptions are not free. Adequacy (Assumption 1) is validated by testing, not proof; cross-validation of a critical property against a second, independent EVM semantics [3] is the strongest available mitigation. Checker soundness (Assumption 3) inherits the trust placed in the proof checker and backend; a small, auditable certificate checker is preferable to a monolithic decision procedure precisely because it shrinks this assumption. Artifact identity (Assumption 4) sounds trivial yet is a frequent practical failure when a verified artifact and a deployed artifact differ by metadata or constructor handling.

7.3 Evaluation directions

Soundness non-violation—no accepted pair fails an independent re-check, including against a second semantics—is necessary but not sufficient. A convincing evaluation also exercises the negative space: deliberately buggy contracts, mutated contracts, weakened specifications, fork-mismatch and artifact-mismatch tests, and negative controls for under-specified properties. The conservative property must be checked directly: the system must yield non-acceptance, never a falsely accepted artifact, when the checker returns Unknown.

8 Related Work

The framework sits at the intersection of three lines of work. *Autoformalization* translates informal statements into formal artifacts; its central obstacle—instability under paraphrase [4] and the need to evaluate informal-to-formal alignment explicitly [5], surveyed in [7]—motivates the detect-don't-trust posture adopted here. *Semantics-first verification of EVM bytecode* supplies the sound core and the validated machine semantics on which the soundness theorem rests [1], with independent semantics in other proof systems [3] enabling cross-validation. *Verified compilation* [2] frames both the aspiration and its limit: end-to-end correctness is provable only where both endpoints are formal, which is exactly why the natural-language endpoint must be handled by a human oracle rather than a proof. The framework's distinguishing commitment is to treat the interface itself—the trust boundary—as the object of study.

9 Conclusion of Part I

A proof, not a hallucination, is achieved by a hard boundary between untrusted generation and a trusted, formal core: the generator may produce arbitrary output to the left of the core, and the core admits only what its checker can validate against a sound semantics. By separating the operational checker from semantic truth, the soundness theorem is a genuine result rather than a restatement of a definition; by specifying over full configurations, it covers trace, exit-status, and event properties rather than terminal states alone. No bespoke compiler is required; verifying deployed bytecode removes the source compiler from the trusted base. The single *non-formal* gap—the alignment between specification and intent—is discharged as an explicit human-oracle assumption held outside the formal core, and the residual weakness—specification completeness—is named, equipped with an explicitly heuristic coverage proxy, and set out as the principal direction for future work. The core claim is narrow and defensible: *system soundness is invariant to the capabilities of the generative model.*

Part II
Proof-Carrying Bundles for Critical Infrastructure (version 3)

10 From Pipeline to Bundle

The unit of trust in critical infrastructure is not a pipeline but an artifact that crosses a boundary. A contract is deployed, a firmware image is flashed, a kernel module is loaded, a circuit is published; at that moment the producer and its tooling are gone, and whoever runs the artifact must decide whether to trust it with nothing but the artifact in hand. Part I answered the question *how can an untrusted generator produce a trusted artifact?* Part II asks the question that follows: *how does that trust travel with the artifact to a party that never saw the generator?*

The answer in Part I was already implicit. Acceptance was a decidable, syntactic check of a certificate c against a specification φ and an artifact b over a formal semantics. Everything the checker needed was an input; nothing about the producer entered. But the process returned (b, φ) and discarded c, so a second party had to re-run synthesis or take the first party's word. Both options reintroduce the trust the framework was built to remove. The fix is to return the inputs of the check, not its verdict.

Part I reads as a paper on EVM verification with a generator in front of it; the EVM semantics, forks and fee schedules occupy the definitions. In Part II the EVM is a parameter. What remains when it is abstracted away is a general statement about any domain in which correctness is formalizable: there exists a bundle format such that the producer may be arbitrary, adversarial or absent, and the consumer still decides acceptance with a small trusted base. This is the thread the paper now follows end to end.

Three commitments carry through from Part I unchanged. The generator is untrusted and stays untrusted; nothing in Part II makes it more reliable, and nothing needs to. Soundness is a theorem relative to an explicit trusted base, not a property of a tool. And the alignment between formal specification and human intent is a non-formal gap discharged by a human oracle, not hidden. Part II adds one commitment: every assumption in the trusted base that can be turned into a mechanical check is turned into one, and every assumption that cannot is written into the bundle as data the consumer can inspect.

Contributions of Part II.

1. A domain-parametric restatement of TCG in which the EVM is one instantiation of a verifiable domain (Section 11).
2. The proof-carrying bundle: a content-addressed object carrying artifact, specification, certificate, instantiated trusted base and evidence, with a re-verifiability proposition (Sections 13–14).
3. A verdict ladder that reports kernel-checked, judged and informative layers separately, so that a bundle never presents an opinion as a proof (Section 15).
4. A treatment of the generation session, including generator traces, as provenance that is recorded but never trusted, resolving the tension between auditability and capability-invariance (Section 17).

11 Verifiable Domains

A domain admits a bundle exactly when it supplies five things: configurations, a formal transition relation validated against the real machine, a specification language over traces, a certificate language with a sound checker, and an identity function that ties the verified artifact to the deployed one. The EVM supplies all five; Section 18 sketches how several other substrates of critical infrastructure would supply them. Nothing else about the EVM is used. Throughout Part II, "kernel" means the domain's trusted certificate checker: a proof-assistant kernel such as Lean's, or, for an SMT-backed domain, a proof-format checker or a checker that replays counterexamples as ground executions.

Definition 7 (Verifiable domain)**.** A verifiable domain is a tuple

$$\mathscr{D} = (\Sigma,\ \mathscr{E},\ \to_{\mathsf{M}},\ \to_{\mathscr{D}},\ \mathsf{Obs},\ \mathrm{Exit},\ \mathsf{Spec},\ \mathscr{C},\ \mathrm{Check},\ \mathrm{id})$$

where Σ is the set of full machine configurations; $\mathscr{E}$ is a family of environment classes, each a set of configurations agreeing on the parameters that make execution deterministic (for the EVM: fork, fee schedule, external code, caller and calldata shape); $\to_{\mathsf{M}}$ is the transition relation of the real machine and $\to_{\mathscr{D}}$ that of the formal semantics, both indexed by an environment class; Obs is the set of externally observable events a trace may emit; Exit is the finite set of terminating statuses; Spec is the set of specifications; $\mathscr{C}$ is the set of certificates; $\mathrm{Check} : \mathsf{Spec} \times B \times A \times \mathscr{C} \to \mathrm{Res}$ is the checker, with B the set of artifacts, A the set of annotations and $\mathrm{Res} = \mathsf{Accept}(\mathscr{C}) \uplus \mathsf{Refute}(CE) \uplus \mathsf{Unknown}$ as in Part I; and $\mathrm{id} : B \to Id$ maps an artifact to the identity under which it is deployed, typically a cryptographic hash of its exact deployed form.

Definition 8 (Specification, domain-parametric)**.** A specification is $\varphi = (\mathrm{Pre}, \mathrm{Trace}, \mathrm{Post}, \mathrm{Exit}^*)$ with Pre, Post predicates over Σ, Trace a predicate over traces (sequences of configurations with their Obs events), and $\mathrm{Exit}^* \subseteq \mathrm{Exit}$. This is Definition 2 with the exit set drawn from the domain. For domains with a single terminating status, Exit^* is trivial and the specification collapses to a trace-sensitive Hoare triple.

Definition 9 (Semantic claim, domain-parametric)**.** For an environment class $E \in \mathscr{E}$, write $\to^E_{\mathscr{D}} \models \varphi(b)$ when every terminating $\to^E_{\mathscr{D}}$-trace of b from a configuration satisfying Pre satisfies φ. Termination is guaranteed in gas- or fuel-bounded domains; otherwise the claim is one of partial correctness, over terminating traces only, and the bundle records which.

Two requirements on the domain are not formal and must be met by evidence. **Adequacy**: $\to^E_{\mathscr{D}}$ agrees with $\to^E_{\mathsf{M}}$; the standard form of evidence is a conformance suite that both relations pass, strengthened by a second, independently written semantics on which critical properties are cross-checked. **Identity**: id is injective on the artifacts that can actually be deployed, so that equality of identities is equality of artifacts. For bytecode and firmware images this is a collision-resistant hash; for domains that deploy source, the compiler enters the trusted base and the bundle must say so.

The boundary of applicability is unchanged from Part I: a task without a correctness predicate ("design a profitable protocol", "write a fair governance rule") has no φ and admits no bundle. What Part II adds is that within the boundary the format is uniform. A consumer who can check one domain's bundle needs to learn nothing domain-specific to check another's; only the semantics and the checker binary change, and both arrive inside the bundle by hash.

12 Trust Classes

Every component and every artifact in Part II, other than a checked fact, carries one of three trust statuses, and the bundle makes the status of each part explicit rather than letting the reader infer it. The classes are those of Section 3.3; what changes is that they now label data in the bundle as well as components in the pipeline (Table 1).

The principle governing the table is that a component's output inherits the class of the component only until it is checked. A certificate written by an untrusted synthesizer is U while it is being written and T once Check accepts it, because acceptance is a property of the certificate, not of its author. A counterexample from an untrusted symbolic executor is U until it is replayed under $\to_{\mathscr{D}}$, at which point it is a concrete trace under $\to_{\mathscr{D}}$ and belongs to no class at all: it is a checked fact. The bundle records both the origin and the moment of promotion. The evidence layer is labelled T^* for this reason: its contents are produced by U tools, those that can be replayed become checked facts, and the conclusions drawn from them about φ are H's. The non-vacuity witness is one object, stored in $\hat{T}$ and referenced from V.

The second principle is that T^* material never enters Check. Back-translations, mutant results and approvals are inputs to the human's decision, and the bundle carries them so that the decision is auditable; they are never inputs to the kernel. This is what keeps capability-invariance intact. Proposition 1 proved

Class	Meaning	Pipeline components	Bundle contents
U untrusted	May behave arbitrarily, including adversarially; nothing it produces is believed until checked	Formalizer F, generator G, invariant synthesizer I, any LLM, any symbolic executor producing counterexamples	Provenance layer: session ledger, iteration log, generator traces, model attestations
T trusted-formal	Belongs to the formal core; its correctness is an explicit assumption, and its size is named	Check, the certificate checker, the formal semantics $\rightarrow_{\mathscr{D}}$	Hard layer: b, φ, a, c and $\hat{T}$, which holds the semantics and the checker by hash
T^* trusted non-formal	Sole bearer of meaning that no formal artifact can carry; cannot be replaced by a proof	Human oracle H	Evidence layer: signed approvals of φ, back-translations, mutant witnesses, coverage indicators

Table 1: Trust classes, applied to components and to bundle contents.

that replacing F, G, I with adversaries leaves soundness unchanged; Part II extends the claim to the evidence and provenance layers. Replacing every piece of evidence in the bundle with fabrications leaves the hard layer's verdict unchanged, and the verdict ladder (Section 15) is designed so that such fabrication can only lower a bundle's reported level, never raise it.

A consequence for the question of *why the generator chose what it chose*: the answer, in whatever form it is recorded, is U. Mechanistic traces of an open-weight model, attribution over the specification, ranked alternatives and counterfactual re-runs are useful to the human who must judge φ and to the engineer who must improve completeness, and they are placed in the provenance layer for exactly those readers. They have no bearing on acceptance and the bundle format forbids them from acquiring one.

13 The Proof-Carrying Bundle

A bundle is the complete set of inputs to Check, plus the instantiated trusted base and the evidence on which the human oracle acted, bound into one content-addressed object. Nothing in it is a verdict; verdicts are recomputed by whoever holds it.

Definition 10 (Bundle)**.** A bundle over a verifiable domain $\mathscr{D}$ is

$$\beta = (b,\ \varphi,\ a,\ c,\ \hat{T},\ V,\ P)$$

where b is the artifact in its deployed form; φ the specification; $a \in A_{\text{chk}}$ the checked annotations; $c \in \mathscr{C}$ the certificate; $\hat{T}$ the instantiated trusted base; V the evidence layer; and P the provenance layer. The first five are the hard layer. $\hat{T}$ is the data form of Assumptions 1–6: the hash and version of the formal semantics, the environment class E as a machine-readable description, the exact A_{asm} facts with their justifications, the identity $\text{id}(b)$, the satisfiability witness for Pre, and the hash of the certificate checker itself. V and P are described in Sections 16 and 17.

Structure. The bundle is a Merkle DAG, not an archive. Each component is a node addressed by the hash of its content; the manifest is the root and holds, for each child, its hash, its trust class and its media type. Three properties follow. A consumer may fetch a subset of nodes and verify them against the root without fetching the rest, so a kernel check needs b, φ, a, c, $\hat{T}$ and the checker and nothing else. Any change to any component changes the root, so a bundle cannot be edited after signing. And identical components across bundles deduplicate, so the formal semantics of a domain, several megabytes, is stored once per consumer and referenced by every bundle in that domain. Figure 1 shows the structure: only the left column is an input to the checker; the manifest records a trust class for every child so that a reader cannot mistake evidence or provenance for proof.

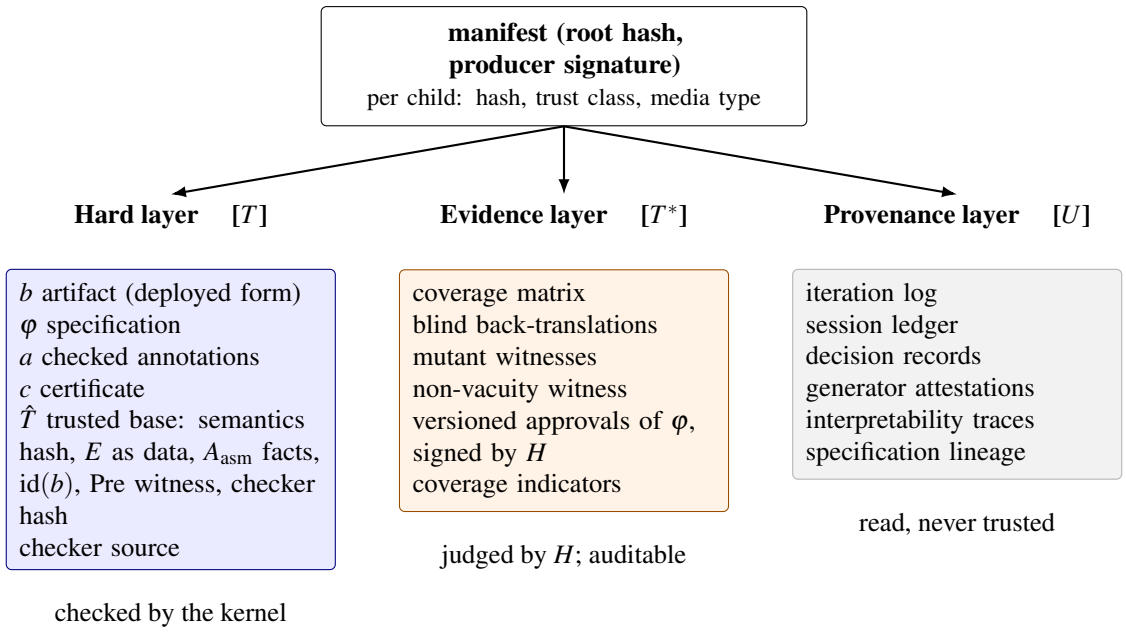


Figure 1: Bundle structure: one root, three layers, three trust classes. The kernel checks the hard layer; H judges the evidence layer; the provenance layer is read, never trusted.

Signatures. The root is signed by the producer. The approval of φ inside V is signed separately by H. These are different keys with different meanings: the producer's signature says *this is what I assembled*; the oracle's signature says *this specification is the one I accepted as the meaning of the intent*. A bundle whose root signature is valid but whose approval signature is missing or belongs to the producer is well-formed and reports as such on the ladder: the hard layer stands, the T^* layer does not.

The checker travels with the bundle. $\hat{T}$ includes the source and hash of a certificate checker for $\mathscr{D}$ that is small enough to audit and self-contained enough to build from the bundle alone. For a proof-assistant-based domain this is an exported-declaration checker of a few thousand lines, not the proof assistant that produced the declarations; for an SMT-based domain it is a proof-format checker, not the solver. The bundle therefore does not depend on the continued existence, availability or compatibility of the producer's toolchain. The consumer builds the checker, confirms its hash matches $\hat{T}$, and runs it. A bundle that can only be checked with tools outside itself is not self-contained and is reported as such.

What is deliberately absent. The bundle does not contain the generator or its weights, holds its prompts only as provenance, and makes no claim about the quality of the generation process. Those records belong to P, are informative, and are discussed in Section 17. The bundle also does not contain the human's reasoning for approving φ, only the fact and the inputs. Part I's position that H cannot be replaced by a formal artifact is preserved by not pretending to formalize H.

14 Re-verifiability

The central result of Part II is this: a bundle is sufficient for a third party to recompute acceptance, and two of the seven assumptions of Section 4 become mechanical checks on the way.

Proposition 2 (Re-verifiability)**.** *Let $\beta = (b, \varphi, a, c, \hat{T}, V, P)$ be a bundle over $\mathscr{D}$ and let* $\mathrm{Check}_{\hat{T}}$ *be the certificate checker whose source and hash $\hat{T}$ records. Any party that builds* $\mathrm{Check}_{\hat{T}}$ *from β, confirms its hash and runs it on (φ, b, a, c) obtains the same verdict, and that verdict is computable from β alone.*

Proof. Builds of $\text{Check}_{\hat{T}}$ that match the recorded hash are the same program. Validating a given certificate is a deterministic procedure on its inputs; Unknown arises in the producer's search for a certificate, not in the validation of one. The four inputs and the semantics are components of β, fixed by hash. No step references F, G, I, the session, or any material outside β. □

The proposition is trivial, and its triviality is the design goal: if re-verification required anything not in the bundle, the bundle would be incomplete. The engineering content is in making the inputs self-contained, which is what Section 13 specifies. The producer may have accepted the certificate with a full proof assistant, which is a different program; the verdict that counts is the one $\text{Check}_{\hat{T}}$ returns, and a bundle that the producer's toolchain accepts but $\text{Check}_{\hat{T}}$ rejects does not pass L1.

Theorem 2 (Soundness, transported)**.** *Under Assumptions 1, 5 and 6 instantiated by $\hat{T}$, Assumption 7 (whose signed approval is carried in V), and with Assumptions 2 and 4 checked as below, if some checker* Check' *satisfying Assumption 3 for $\mathscr{D}$ returns* $\text{Check}'(\varphi, b, a, c) = \mathsf{Accept}(c)$ *for a bundle β, then every real $\to_{\mathsf{M}}^{E}$-trace of the deployed artifact from a reachable* Pre*-configuration satisfies φ.*

Proof. As in Theorem 1, with the checker replaced by Check' and the deployed artifact identified with b by the identity check below. □

The hash match of Proposition 2 is not needed for Theorem 2: soundness of whichever checker accepted is enough. It matters for reproducing the verdict, which is why L1 reports which checker accepted (Section 15). Two sound checkers can still disagree, since soundness constrains only acceptance: a second, independently written checker that accepts gives the conclusion of Theorem 2 on its own, and one that rejects what $\text{Check}_{\hat{T}}$ accepts signals a bug in one of the two.

14.1 Assumptions that become checks

Section 4 listed artifact identity (Assumption 4) and environment adequacy (Assumption 2) as assumptions because the process ended before deployment. The bundle outlives deployment, so the consumer is in a position to verify both (Assumption 2 only for the parameters that E declares; see Section 19).

Identity. $\hat{T}$ contains $\text{id}(b)$. The consumer computes id of the artifact actually present in the deployment target (the code at an address, the image in a partition, the module in a loaded table) and compares. For the EVM this is the Keccak-256 hash of the runtime bytecode at the address against the constant in the Lean or K development (Lean developments include [18, 19]), with the constructor and immutables handled as $\hat{T}$ specifies. Inequality is not a weaker verdict; it is a different artifact, and the bundle says nothing about it. This turns the assumption Section 7.2 flagged as a practical failure into the cheapest check in the ladder.

Environment. $\hat{T}$ encodes E as data: chain identifier and fork, fee schedule version, assumptions on callers and external code, precompile set, resource limits. The consumer compares it with the live deployment environment. A mismatch does not invalidate the proof; it invalidates the transfer from $\to_{\mathscr{D}}$ to $\to_{\mathsf{M}}$ for that deployment, and the ladder reports exactly that.

What stays an assumption. Adequacy of the semantics (Assumption 1) and soundness of the checker (Assumption 3) cannot be checked by the consumer; they are the irreducible trusted base. $\hat{T}$ names them with hashes and sizes so that the consumer knows exactly what they are trusting. The consumer can substitute an independently audited semantics or checker; the soundness conclusion then rests on that substitute's Assumptions 1 and 3, and L1 reports which was used. Specification well-formedness (Assumption 6) is partly checkable: $\hat{T}$ carries a satisfiability witness for Pre, which the consumer replays. Reachability of that witness in the deployment remains a matter of judgment. Annotation discipline (Assumption 5) is checkable in that every A_{asm} fact must be listed in $\hat{T}$; the justification of each is not.

15 The Verdict Ladder

A bundle is never summarized by one bit. The consumer's verifier reports each of L0–L4 separately as passed or failed. The levels are not ordered by strength: L0 and L1 are mechanically decided, L2 and L3 bear on the human's judgment, and L4 is informative only (Table 2). They have dependencies: L1 presupposes L0, L2 and L3 presuppose L1, and L4 is independent of the rest.

Level	Statement	Decided by	Trust class
L0	Root hash and producer signature valid; $\mathrm{id}(b)$ equals the deployed artifact; E matches the deployment	Hash comparison	—
L1	$\mathrm{Check}_{\hat{T}}$ accepts c, or a consumer-chosen sound checker does and is reported as such; no admitted holes; only the domain's standard axioms used	Certificate checker	T
L2	Every requirement of the intent n maps to ≥ 1 clause of φ; every clause maps back; back-translation of φ agrees with the informal text; known-bad mutants are refuted; Pre witness replays	Tools run by the consumer, judged by H	T^*
L3	H's approval of this exact φ is present and signed; the approved version is the proven version	Signature + hash equality	T^*
L4	P is present, internally consistent, and the tool invocations it records replay to the hashes it claims	Replay of U tools	U

Table 2: The verdict ladder.

L0 and L1 are the hard layer and are the only levels that correspond to a theorem. A consumer who stops at L1 holds a kernel-checked proof that the deployed artifact satisfies φ under the stated trusted base, and knows nothing about whether φ is the right specification.

L2 and L3 concern the single non-formal gap. L2 reports that the tools which help a human judge alignment were run and what they returned; it does not report that alignment holds, because no tool can. L3 reports that a human took responsibility for exactly the φ that was proven, by signing its hash. A mismatch between proven φ and approved φ is a real failure mode: a specification is approved, then weakened during synthesis to make the proof go through, and the weakened version is what ships. L3 fails in that case even though L1 passes.

L4 concerns provenance and is purely informative. A bundle that passes L4 tells the consumer how the artifact came to be and lets them replay the deterministic parts of that history. It adds nothing to the correctness claim and the ladder's ordering encodes that: no amount of provenance can make L2 or L3 pass, and fabricating provenance can only cause L4 to fail.

Monotonicity. The ladder is designed so that adding fraudulent material to a bundle can make a level fail and never make it pass. This holds for L0, L1, L3 and L4, which are decided by recomputation or by signature verification against keys the consumer chooses to trust. It holds for L2 only if the consumer supplies or regenerates the mutant set and the back-translations: a producer-chosen set of easy mutants, duly refuted, would make L2 pass. The held-out library of the companion note [9] is designed for this case. Within that scope, this is the bundle-level form of capability-invariance: the producer may be arbitrary, and the only thing an arbitrary producer can do to the verdict is make it worse.

Reporting. The verifier emits the ladder as structured output alongside the hashes it compared and the axioms it found, so that a second tool or a model can read the result without re-running the check. A consumer-facing summary that collapses the ladder to a single mark is a misrepresentation of the bundle, and a verifier conforming to this format does not produce one.

16 The Evidence Layer

The evidence layer V exists because a correct proof of the wrong specification is the main remaining failure mode once the hard layer is in place. V does not close the gap between φ and intent; it makes the human's closing of it inspectable and gives the human instruments that reduce the chance of a false approval. Everything in V except H's signed approvals is produced by untrusted tools and consumed by H.

Coverage matrix. The intent n is split into numbered requirements. Each clause of φ carries references to the requirements it is meant to express; each requirement lists the clauses that express it. A tool checks that the matrix is total in both directions. A requirement with no clause is a declared blind spot (Corollary 1) and is listed as such; a clause with no requirement is either a strengthening the human should know about or a leak of implementation detail into the specification.

Blind back-translation. An untrusted model that has not seen n translates each clause of φ into natural language. A second step compares the translation with the requirement it references. Disagreement is reported, not resolved; the human decides. The method is cheap and is meant to catch common autoformalization errors of quantifier scope, off-by-one boundaries and silently dropped conditions, which are invisible in the formal text to a reader who already believes they know what it says.

Mutant witnesses. For each clause, V contains at least one artifact b' that violates the intended property together with a $\mathsf{Refute}(ce)$ from Check or a concrete trace under $\to_{\mathscr{D}}$ showing the violation. A clause that no mutant in the declared set violates is vacuous or too weak relative to that set, which is the caveat attached to Proposition 1. The witnesses are replayable: the consumer runs the trace and confirms the clause fails. This part of V is mechanically decisive, and it decides weakness, never adequacy. The companion note [9] develops this device into a witness-based strength order on specifications, a certified mutation score recomputed by the same checker, and a gate that can be used as a reward for a specification-writing generator.

Non-vacuity witness. A concrete configuration $\sigma_0 \models \mathrm{Pre}$, stored in $\hat{T}$ and referenced from V, replayable by the consumer, so that the guarantee is known to quantify over a non-empty set. It shows that Pre is satisfiable; it does not detect vacuity from unreachable branches or dead code inside b, which the mutant witnesses address only for the faults they inject.

Versioned approval. The specification is versioned. Each approval by H is a signature over the hash of a specific version and the hash of the requirement list it was checked against. The bundle carries the chain of versions from the first the human saw to the one that was proven. L3 requires that the last approved hash equals the proven hash; the chain lets an auditor see what changed between approvals and whether a weakening happened after the last one.

Coverage indicators. Definition 6 and its companion metrics (mutation score, obligation coverage, exit-status coverage, boundary coverage, adversarial-property checklist) are reported as numbers with their computation method named. None is a completeness measure and the bundle labels each as a heuristic. Their value is comparative: two bundles for the same requirement list can be ranked, and a drop between versions is a signal.

What the evidence layer refuses to contain. V does not contain a claim that φ matches n, in any form, from any tool. A model's statement *this specification captures the intent* is an opinion of a U component about a T^* question and has no place in the bundle outside the provenance layer. The design accepts that this makes V look weaker than a confident summary would; it is weaker.

17 The Provenance Layer

The provenance layer P records how the bundle came to exist and is the only place the generation session appears. It is informative by construction: nothing in P is an input to Check, to the ladder below L4, or to the human's approval of φ as a formal matter. Its readers are auditors reconstructing a decision, engineers improving specification completeness, and future producers reusing a lineage.

Iteration log. The process of Definition 5 runs up to $K_{\max}$ iterations and discards everything but the accepted one. P keeps the sequence: for each k, the hashes of b_k, a_k and c_k, the checker's result, and for Refute the counterexample ce_k. This record is a direct input to specification completeness. Every Refute shows a path φ constrains; every Unknown shows a region where the checker could not decide, and Unknown regions are the natural candidates for both a stronger annotation and a missing requirement. Part I identified completeness as the central open problem; the iteration log is the cheapest data source for it and Part I threw it away.

Session ledger. Every tool invocation during the session (compiler, test runner, symbolic executor, checker, formatter) is recorded as an event with the hashes of its inputs, its outputs and the tool's own identity. The ledger is append-only and hash-chained. L4 replays the deterministic events and confirms the hashes. Generator outputs are recorded as events too but are not replayed; the model is not reproducible and the bundle does not pretend it is. The ledger answers *what ran, on what, and what came out*, which is the question an auditor asks first.

Decision records. At each design decision the producer deems significant, P holds a record: the alternatives considered, the one taken, and the evidence that discriminated them. The evidence must be a reference into the ledger (a test that failed on the alternative, a counterexample, a checker result), not prose. A decision record whose only support is the generator's stated reasoning is permitted but tagged as unsupported. This is where the answer to *why this and not that* lives, and the format forces the answer to point at a fact or admit it cannot.

Generator attestations. For a closed-weight generator: the provider's signature over model identifier, input hash, output hash and sampling parameters, when the provider offers one. For an open-weight generator: the weight hash, tokenizer hash, inference configuration and hardware fingerprint needed to reproduce activations. Neither says anything about correctness. Both say which untrusted component produced a given output, which matters when a defect is traced back and a producer wants to know whether to stop using a model.

Interpretability traces. For open-weight generators, P may carry compressed mechanistic records at decision points: sparse feature activations, attribution graphs for the token at which a decision was made, and counterfactual interventions showing that suppressing a feature changed the output. For any generator it may carry input attribution: which requirements of n, when removed, flip the decision. These are the most informative and least trustworthy objects in the bundle. Feature labels are themselves generated by a model; the faithfulness of a reasoning trace to the computation that produced it is not guaranteed; and none of it can be replayed by a consumer without the weights. P tags each trace with its reproducibility class and the ladder never reads them. Their value is to the human reviewing a decision record and to the engineer asking why a model keeps weakening the same clause.

Specification lineage. The chain of φ versions, with the ledger events between approvals. Together with the versioned approvals in V, this makes visible the moment a specification was weakened to admit a proof, which is the failure L3 is built to catch and P is built to explain.

The resolution of the auditability tension. Capability-invariance (Proposition 1) says the generator's internals do not matter. The practical demand for an answer to *why* says they do. Both are right about different questions. The internals do not matter to acceptance, and P is kept out of the hard layer so that remains true. They matter to the humans who must decide whether φ is right and whether the process should be trusted again, and P is kept in the bundle so they can be examined.

18 Instantiations

The EVM is the reference instantiation because it is the easiest: deterministic once the environment is fixed, gas-bounded, with an executable semantics validated against an official conformance suite and a second independent semantics for cross-checking. Table 3 lists it beside other substrates of critical infrastructure that supply the five things listed at the start of Section 11, with the point at which each departs from the EVM case. Part II specifies a format; rows other than the EVM are proposed instantiations, not implemented ones.

Domain	**Configurations and environment**	**Exit statuses**	**Certificate and checker**	**Identity** $\mathrm{id}(b)$	**Where it differs from the EVM**
EVM bytecode	World state, tx and block parameters, fork, fee schedule, external code	halt, revert, oog, exc	Exported Lean declarations [18, 19], or a K proof object once the K prover emits one [17]; external kernel checker	Keccak-256 hash of runtime bytecode at the address, constructor and immutables handled per $\hat{T}$	Reference case
WebAssembly modules	Linear memory, tables, globals, host imports as environment	trap, return	Proof terms over a mechanized Wasm semantics [26]; kernel checker	Hash of the binary module	Host imports are the environment; E must enumerate their contracts, and adequacy evidence is per host
eBPF programs	Kernel context struct, maps, helpers as environment	exit code, reject	Proof that the program satisfies a property stronger than the in-kernel verifier's (cf. [27]); certificate checker	Hash of the bytecode as loaded	The deployment target already runs a checker; the bundle's claim must be stated relative to it, not instead of it
Zero-knowledge circuits	Witness and public inputs; the constraint system is the artifact	satisfied, unsatisfied	Proof that the constraint system is equivalent to a reference function (soundness and completeness of the circuit)	Hash of the verifying key or constraint system	Both directions matter: an under-constrained circuit accepts bad witnesses and is the typical defect [28]; Trace is trivial, Pre/Post carry everything
Firmware for a fixed ISA	Registers, memory, device state as environment; interrupts as events	halt, fault	Proof over an executable ISA semantics; kernel checker	Hash of the image in the partition	Termination is not bounded by fuel; the bundle records partial correctness and names the liveness assumption
PLC safety logic	I/O image, timers, cycle	end of cycle, fault	Proof of cycle invariants over a semantics of the control language	Hash of the compiled program on the controller	Properties are invariants over an unbounded run of bounded cycles; Trace carries the interlock discipline

Table 3: Instantiations of the verifiable domain. Rows other than the EVM are proposed.

Two observations cut across the rows. First, the dimension along which domains vary most is the environment class E, not the semantics; the bundle's decision to encode E as data the consumer compares with the deployment (Section 14) does the most work outside the EVM. Second, in every row the checker that travels with the bundle is smaller than the tool that produced the certificate, and in every row the deployed form of the artifact has a canonical hash. These two properties are what make the format uniform.

The EVM instantiation of Part I is recovered by taking $\mathscr{D}$ to be the first row, with Exit $= \{\mathsf{halt}, \mathsf{revert}, \mathsf{oog}, \mathsf{exc}\}$, E parameterized by fork and fee schedule, and Check backed by the formal semantics validated in Assumption 1. Definitions 1, 2 and 4 are then special cases of Definitions 7–9, and Theorem 2 specializes to Theorem 1.

19 Threats and Limits

The bundle shrinks the trusted base and moves most of it into data; it does not remove it. The following are ways a bundle that passes L1 can accompany a defective artifact, and what the format does about each.

Wrong specification. The dominant risk, unchanged from Part I. A proof of φ says nothing about $\psi \notin \mathit{closure}(\varphi)$. The format's response is L2 and L3: tools that make a wrong φ more visible and a signature that makes a human responsible for it. It cannot do more.

Semantics mismodels the machine. Assumption 1 is validated, not proved. A property true under $\to_{\mathscr{D}}$ and false under $\to_{\mathsf{M}}$ is undetectable from inside the bundle. Mitigation is cross-checking critical clauses against a second semantics with no shared code and recording both checker hashes in $\hat{T}$. A consumer who distrusts a semantics can substitute another of the same interface and re-run; the bundle format makes that substitution a hash change, not a reconstruction.

Checker unsoundness. Assumption 3. The bundle ships a small checker precisely so that this assumption is auditable; a consumer who audits it once can trust every bundle in the domain that records its hash. A bug in the checker that accepts a malformed certificate can be caught by running a second independent checker of the same format, which the bundle's content addressing makes cheap.

Custom axioms and admitted holes. A certificate may introduce an axiom that makes everything provable. L1 requires the checker to enumerate the axioms used and compare against the domain's standard set, and to reject any admitted obligation. This is a mechanical check, and it targets a common defect in machine-generated proofs; it is listed under threats because a verifier that omits it passes bundles that prove nothing.

Artifact substitution. The bundle is about b; the deployment holds b'. L0 catches this when the consumer actually computes id of the deployed artifact rather than trusting a hash the producer supplied. The threat is procedural: a verifier that reads $\mathrm{id}(b)$ from $\hat{T}$ and compares it with $\mathrm{id}(b)$ from $\hat{T}$ passes everything.

Environment drift. A proof under fork f consumed under fork f'. L0 catches a declared mismatch. An undeclared one, where E in $\hat{T}$ is silent about a parameter that changed, is a specification-completeness failure in $\hat{T}$ rather than in φ, and the mitigation is the same: enumerate what E must mention per domain and reject $\hat{T}$ that omits a required field.

Cross-artifact properties. A bundle is about one artifact. Reentrancy across two contracts, storage collision under delegated calls, a firmware image and the bootloader that loads it: none is in any single φ unless the environment models the other party explicitly. The format allows a bundle to reference another bundle by root hash as part of its E, which lets composed guarantees be stated, but composition is not automatic and Part II does not provide a composition theorem.

Forged evidence and provenance. V and P can be fabricated by the producer. Within the scope stated in Section 15, the ladder's monotonicity prevents this from making a level pass; it can still mislead a human reading P. The defense is the rule that decision records must reference ledger events, and that ledger events replay; a fabricated chain of events that replays to the claimed hashes is a real chain of events.

What the bundle does not claim. It does not claim the artifact is secure, only that it satisfies φ. It does not claim φ is complete. It does not claim the generator is good, or that its reasoning was sound, or that its traces are faithful. It does not claim economic or game-theoretic properties of the system the artifact participates in. It does not claim anything about an artifact whose identity differs from $\mathrm{id}(b)$ by one byte. Each of these is a sentence in the manifest's `not_claimed` field, present in every bundle, because the absence of a claim is the thing a reader is most likely to miss.

20 Related Work for Part II

The bundle is an instance of proof-carrying code [10]: the producer ships a certificate and the consumer checks it with a small trusted checker, never trusting the producer's tools. Foundational proof-carrying code [11] moved the machine semantics itself into the checked base, which is the role $\hat{T}$ plays here, and translation validation [12] checks each output of an untrusted transformation instead of verifying the transformation, which is the posture TCG takes toward the generator. Small certificate checkers exist for SAT (DRAT and LRAT [13, 14]), for SMT (the Alethe format [15]) and for proof assistants, where an independent typechecker re-checks exported declarations [16]; these are the checkers Section 13 asks to travel with the bundle. For the K framework, proof generation in matching logic [17] is the route by which a K-based EVM proof could become a checkable certificate.

Closest to the EVM instantiation, EquiVM [18] obtains foundational Lean proofs, generated by LLMs, that deployed EVM bytecode refines its specification, each proof serving as a per-artifact certificate; EVMYulLean [19] is an executable Lean model of the EVM validated against the Ethereum conformance tests. Both are candidates for the semantics and certificate of the EVM row of Table 3. Clover [20] combines LLM generation with consistency checks among code, docstrings and formal annotations in Dafny, a different answer to the question of when a generated artifact can be trusted.

The evidence layer draws on vacuity detection in model checking [21] and on mutation-based assessment of specifications [22]; the companion note [9] surveys that line and builds the mutant witnesses into a certified score. The provenance layer follows software supply-chain practice: in-toto [23] records signed steps of a build, Sigstore [24] binds signatures to identities, and reproducible builds [25] make a binary re-derivable from its source. A supply-chain attestation says who produced an artifact and how; the hard layer of a bundle says, checkably, what the artifact does, and its provenance layer is kept out of that verdict. Per-domain tools named in Table 3 include mechanized WebAssembly semantics [26], the PREVAIL verifier for eBPF [27], and under-constraint detection for zero-knowledge circuits [28].

21 Changes from Version 2 and Open Problems

Version 3 keeps every definition, assumption and result of version 2 and changes where they live. Table 4 lists each change and the section that carries it.

Version 2	Version 3	Section
EVM configurations, forks and fee schedules in the definitions	Verifiable domain $\mathscr{D}$ as a parameter; EVM as its reference instantiation	11, 18
Process returns (b, φ); certificate discarded	Process returns bundle β; certificate, annotations and trusted base retained	13
Soundness relative to the producer's checker	Re-verifiability: the checker recorded by hash in $\hat{T}$ recomputes the verdict from β alone; soundness needs only that some sound checker accepted	14
Assumption 4 (artifact identity)	Mechanical check at L0 against the deployed artifact	14, 15
Assumption 2 (environment adequacy)	E encoded as data in $\hat{T}$ and compared with the deployment at L0	14, 15
Refute(ce) consumed and discarded	Iteration log in P; counterexamples and Unknown regions kept as completeness signal	17
One φ, approved once	Versioned φ; each approval a signature over a hash; L3 requires approved hash = proven hash	16
Trust classes label components	Trust classes label bundle contents; verdict ladder reports each layer separately	12, 15
Heuristic coverage proxy	Same proxies, reported in V with method, explicitly comparative	16
Generator internals out of scope	Generator internals in P, informative only, with the auditability tension resolved by layer separation	17

Table 4: Changes from version 2 to version 3.

Errata to Part I. Part I reproduces version 2 without correction. The following errors are known and are recorded here rather than fixed in place.

1. Section 7.1 says a wrong or too-weak invariant "is refuted". A failed preservation or exit obligation yields a counterexample to induction, which may start in an unreachable state; the claim is not refuted but not accepted, and Section 3.4 reserves Refute for concrete counterexample traces.
2. The abstract of version 2, Remark 2 and Section 9 say the non-formal gap and Assumptions 1–4 are "discharged". In this paper "discharge" means proving an obligation; those gaps are stated as assumptions, not discharged.
3. Section 3.3 uses N, S, B and CE without defining them: they are the sets of informal intents, specifications, bytecode artifacts and counterexample traces.
4. Definition 2 calls the case Trace $\equiv$ *true*, Exit $= \{\mathsf{halt}\}$ the partial-correctness terminal claim. It requires every terminating run from Pre to halt successfully and is better called the successful-termination claim.
5. Reference [8] is listed but not cited in Part I; it is the repository of the semantics cited as [1].

Open problems.

1. *Composition.* A bundle is about one artifact. A composition rule that derives a bundle for a system from bundles of its parts, with E of each referencing the others by root hash, is the natural next result and is not provided here. The difficulty is not the proof but the specification language: cross-artifact Trace predicates have no agreed form.

2. *Specification completeness.* Unchanged from Part I as the central open problem. Part II contributes data sources (iteration log, Unknown regions, lineage) and no method. A method that proposes candidate clauses from Unknown regions and routes them through V to H would be the first use of the provenance layer as more than a record.

3. *Checker minimality per domain.* The format assumes a checker small enough to audit. For proof-assistant-based domains this exists; for SMT-backed domains the proof formats are less settled and the

checkers larger. A survey of per-domain minimal checkers, with sizes, would make $\hat{T}$ comparable across domains.

4. *Oracle provenance.* L3 verifies that a key signed an approval, not that a person did. Binding a signature to a human act without a trusted intermediary is outside this paper; the format leaves a field for an attestation of the signing context and does not specify its contents.

5. *Evidence standards.* V contains back-translations, mutants and coverage numbers with no agreed thresholds. The format reports them; it does not say what is enough. Deployment policies that set thresholds per domain belong in a separate, policy-level document, since the thresholds will change and the format should not. The companion note [9] makes the mutant-witness part of V a recomputable quantity and studies what a generator does to it under optimization pressure; thresholds remain policy.

6. *Reproducibility of interpretability traces.* P tags traces by reproducibility class, and for closed-weight generators the class is *not reproducible*. A provider-side interface that returns feature activations under a signature without exposing weights would move such traces one class up. Whether any provider will offer it is not a research question.

The narrow claim of version 2 was that system soundness is invariant to the generator. The narrow claim of version 3 is that this invariance can be carried in an object small enough to check and complete enough to need nothing else. Anyone may write anything; the bundle is how you know what to check, and the ladder is how you know what you have found.

References

[1] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Roşu. KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, pp. 204–217, 2018. DOI: 10.1109/CSF.2018.00022.

[2] X. Leroy. A Formally Verified Compiler Back-End. *Journal of Automated Reasoning*, 43(4):363–446, 2009. DOI: 10.1007/s10817-009-9155-4.

[3] F. Cassez, J. Fuller, M. K. Ghale, D. J. Pearce, and H. M. A. Quiles. Formal and Executable Semantics of the Ethereum Virtual Machine in Dafny. In *Formal Methods (FM 2023)*, Lecture Notes in Computer Science, vol. 14000, pp. 571–583. Springer, 2023. DOI: 10.1007/978-3-031-27481-7_32.

[4] H. Moore and A. Shah. Evaluating Autoformalization Robustness via Semantically Similar Paraphrasing. arXiv preprint arXiv:2511.12784, 2025.

[5] J. Lu, Y. Wan, Y. Huang, J. Xiong, Z. Liu, and Z. Guo. FormalAlign: Automated Alignment Evaluation for Autoformalization. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[6] C. A. R. Hoare. An Axiomatic Basis for Computer Programming. *Communications of the ACM*, 12(10):576–580, 1969. DOI: 10.1145/363235.363259.

[7] K. Weng, L. Du, S. Li, W. Lu, H. Sun, H. Liu, and T. Zhang. Autoformalization in the Era of Large Language Models: A Survey. arXiv preprint arXiv:2505.23486, 2025.

[8] Runtime Verification, Inc. KEVM: Semantics of the Ethereum Virtual Machine in the K Framework. Software repository, `https://github.com/runtimeverification/evm-semantics`, accessed 2025.

[9] V. Stoliarov. Specification Games: Witness-Based Specification Strength and the Vacuity Gate as a Reward. Manuscript, 29 October 2026.

[10] G. C. Necula. Proof-Carrying Code. In *Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 106–119, 1997.

[11] A. W. Appel. Foundational Proof-Carrying Code. In *Proc. 16th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pp. 247–256, 2001.

[12] A. Pnueli, M. Siegel, and E. Singerman. Translation Validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 1384, pp. 151–166. Springer, 1998.

[13] N. Wetzler, M. J. H. Heule, and W. A. Hunt Jr. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *Theory and Applications of Satisfiability Testing (SAT)*, LNCS 8561, pp. 422–429. Springer, 2014.

[14] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt Jr., M. Kaufmann, and P. Schneider-Kamp. Efficient Certified RAT Verification. In *Automated Deduction (CADE-26)*, LNCS 10395, pp. 220–236. Springer, 2017.

[15] H.-J. Schurr, M. Fleury, H. Barbosa, and P. Fontaine. Alethe: Towards a Generic SMT Proof Format. In *Proc. 7th Workshop on Proof eXchange for Theorem Proving (PxTP)*, EPTCS 336, pp. 49–54, 2021.

[16] M. Carneiro. Lean4Lean: Verifying a Typechecker for Lean, in Lean. In *31st International Conference on Types for Proofs and Programs (TYPES 2025)*, LIPIcs 384, pp. 2:1–2:23, 2026. DOI: 10.4230/LIPIcs.TYPES.2025.2.

[17] X. Chen, Z. Lin, M.-T. Trinh, and G. Roşu. Towards a Trustworthy Semantics-Based Language Framework via Proof Generation. In *Computer Aided Verification (CAV)*, LNCS 12760, pp. 477–499. Springer, 2021.

[18] L. Lazaropoulos and Z. Paraskevopoulou. Foundational Refinement Proofs for Deployed Bytecode, at the Price of Tokens. arXiv preprint arXiv:2607.26306, 2026.

[19] Nethermind. EVMYulLean: A Formal Model of the EVM and Yul in Lean. Software repository, https://github.com/NethermindEth/EVMYulLean, 2026.

[20] C. Sun, Y. Sheng, O. Padon, and C. Barrett. Clover: Closed-Loop Verifiable Code Generation. In *Proc. 1st International Symposium on AI Verification (SAIV)*, pp. 134–155. Springer, 2024. DOI: 10.1007/978-3-031-65112-0_7.

[21] I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh. Efficient Detection of Vacuity in Temporal Model Checking. *Formal Methods in System Design*, 18(2):141–163, 2001.

[22] I. Amaral, A. Mendes, and J. Campos. MutDafny: A Mutation-Based Approach to Assess Dafny Specifications. In *Proc. 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. arXiv:2511.15403.

[23] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *Proc. 28th USENIX Security Symposium*, pp. 1393–1410, 2019.

[24] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: Software Signing for Everybody. In *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2353–2367, 2022.

[25] C. Lamb and S. Zacchiroli. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software*, 39(2):62–70, 2022.

[26] C. Watt. Mechanising and Verifying the WebAssembly Specification. In *Proc. 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pp. 53–65, 2018.

[27] E. Gershuni, N. Amit, A. Gurfinkel, N. Narodytska, J. A. Navas, N. Rinetzky, L. Ryzhyk, and M. Sagiv. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proc. 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 1069–1084, 2019.

[28] S. Pailoor, Y. Chen, F. Wang, C. Rodríguez, J. Van Geffen, J. Morton, M. Chu, B. Gu, Y. Feng, and I. Dillig. Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs. *Proc. ACM on Programming Languages*, 7(PLDI), 2023.